

Crashing and Recovering

Resilience Patterns in a Distributed Go Service

Emily Chen · M.S. Computer Science, Northeastern · February 27, 2026

1. Step I — Piazza Community Sharing
2. Step II — Course Reflection: The Causal Chain
3. Step III — Crash/Recovery Demo (Live Results)
4. Project Proposal Preview

What I observed and contributed to class discussion

What I Shared on Piazza

sharing_homework_6_results 117 week_6_questions 1 parras_paper 117 chapter_2_reading 117 lamports_paper 117 logistics 1 other 117

Note History No history yet

← note #775

hw6 results

Updated 6 days ago by Shixian

Hi all! Sharing my findings from Part 2 — I deployed a product-search microservice on AWS ECS Fargate (0.25 vCPU, 512 MB) and used Locust + CloudWatch to systematically find its performance limits. The service scans 100 of 100,000 in-memory products per request — a fixed-cost computation meant to simulate something like model inference or frame processing that you can't optimize away.

Core Test — Baseline → Saturation on a Single Task

The baseline ran 5 concurrent users for 2 minutes. Median response was ~25ms, 95th percentile at 130ms, 0 failures. CPU sat at only ~3.13% — the service had tons of headroom. The really interesting thing was **memory**: it was completely flat the entire time across all three test scenarios. This immediately told me the bottleneck would be compute, not memory, since the product data is loaded once at startup and never reallocated.

As I ramped up concurrent users, CPU climbed linearly — from 3.13% → 15.9% → eventually spiking to 91.2% at peak load — while memory stayed near zero growth. Response time percentiles started diverging noticeably at the 95th+ levels (130–200ms range at the high end), with the median staying low. This is the textbook pattern for a CPU-bound service: the median holds up because Go's goroutine scheduler handles queuing efficiently, but tail latencies blow out as requests wait for compute.

Exploration — Horizontal Scaling

Since vertical scaling wasn't an option (fixed Fargate task size), the only lever was adding more tasks. I scaled from 1 → 3 tasks, distributing across us-west-2b, 2c, and 2d via an ALB.

The results were clean:

Metric	1 Task	3 Tasks
Avg response	39ms	21ms
Median	18ms	16ms
95th percentile	130ms	54ms
Throughput	~3319 req/s	~3345 req/s
Failures	0%	0%

Average latency dropped nearly in half and the 95th percentile fell from 130ms to 54ms, while throughput stayed roughly constant — meaning the extra capacity was being used to reduce queuing rather than increase req/s, which makes sense since the Locust load was fixed. The ALB target response time spiked briefly

Edit 0 23 Views

What I Learned from Classmates

Unresolved @763_13

Shixian 6 days ago

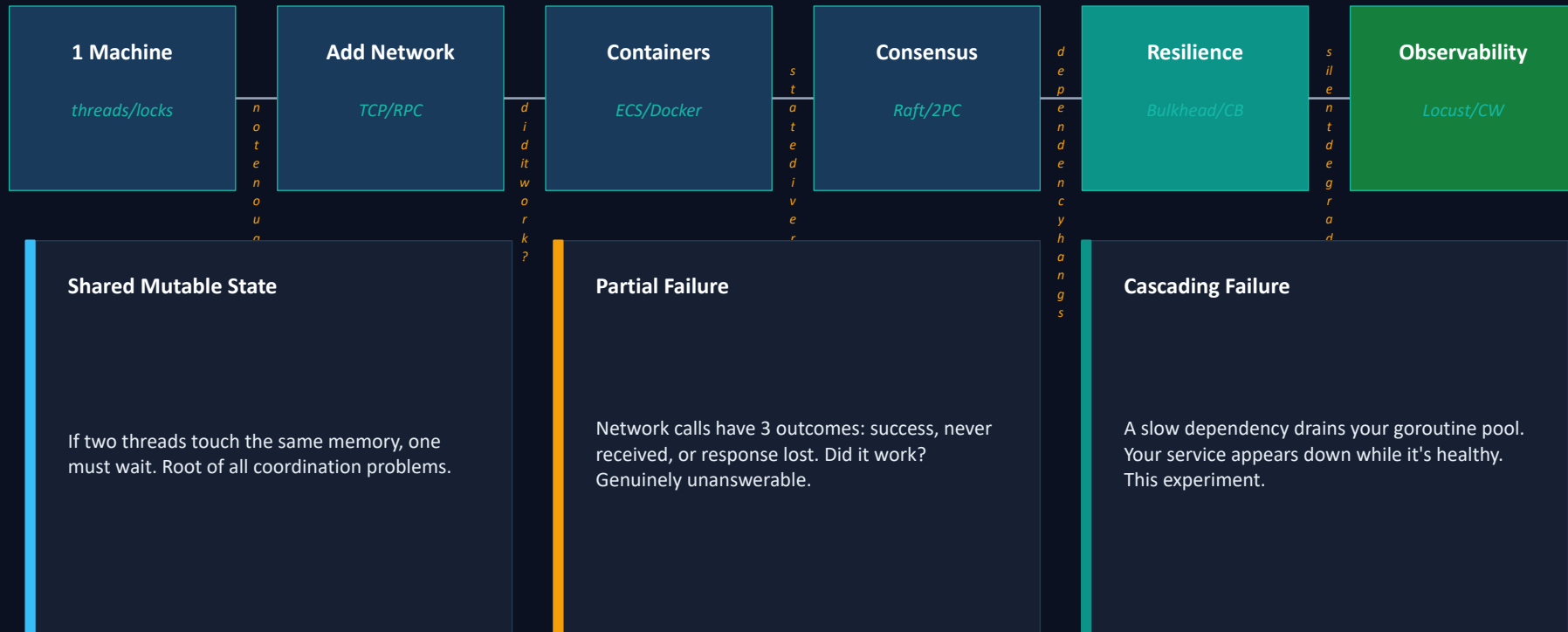
This is a really valuable find — I hadn't considered that zero wait time creates an unbounded demand loop where new capacity just gets immediately consumed. In my tests I used the wait time without thinking much about it, but your post makes clear that it was actually doing important work by capping the total offered load and allowing the system to reach a stable equilibrium. Without it, you're not really testing auto-scaling behavior — you're just measuring max throughput under infinite demand, which is a fundamentally different experiment. The "step down" pattern you described after adding wait time (CPU peaking at 100% then gradually stabilizing below 70% as tasks come online) is exactly what you'd hope to see from a well-tuned scaling policy. Going to keep this in mind for future load tests that the client behavior is just as important to design carefully as the server configuration.

0

Reply

The Causal Chain

Every solution in CS6650 creates the next problem



System Setup

Go Product Search API on AWS ECS Fargate



Component	Technology	Configuration
Web Server	Go net/http	Port 8080 — goroutine per request
Container	Docker Alpine	linux/amd64 single binary
Orchestration	AWS ECS Fargate	2 tasks, auto-scaled
Load Balancer	AWS ALB	Round-robin to 2 targets
Load Testing	Locust (Python)	50 users, spawn rate 5
Monitoring	AWS CloudWatch	ALB metrics + ContainerInsights

What Breaks — and Why

aiRanker hangs 30% of calls for 30 seconds. No protection.

```
broken/main.go

func aiRanker(products []Product) {
    if rand.Float32() < 0.30 {
        // 30% chance of hang
        time.Sleep(30 * time.Second)
        // goroutine STUCK forever ⚠️
    }
    return products
}
```

What Happens Under Load

1. Go spawns a goroutine per request (~2KB each)
2. 30% hit aiRanker → goroutine frozen for 30s
3. 50 users × 30% × 30s = goroutines pile up
4. Memory fills → server chokes → RPS collapses
5. P95 flatlines at exactly 30,000ms

Math: 10 req/sec × 30% hang = 3 stuck goroutines/sec → after 30s: **~90 goroutines frozen simultaneously**

The Broken System

50 Locust users · 5 minutes · unprotected aiRanker

1.3

RPS

30,000ms

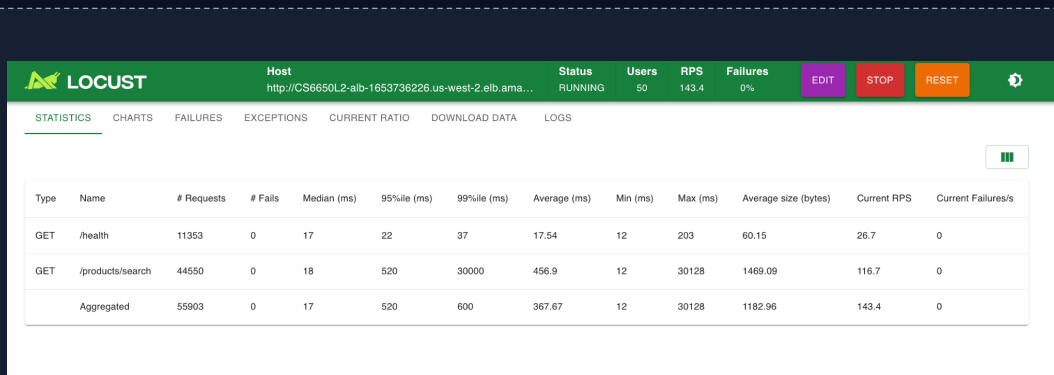
P95 Latency

7,982ms

Avg Latency

0%

Error Rate*



The screenshot shows the Locust web interface with a green header bar. The header includes the Locust logo, host information, status (RUNNING), users (50), RPS (143.4), failures (0%), and buttons for EDIT, STOP, RESET, and a refresh icon. Below the header is a navigation bar with tabs: STATISTICS, CHARTS, FAILURES, EXCEPTIONS, CURRENT RATIO, DOWNLOAD DATA, and LOGS. The STATISTICS tab is active, displaying a table with performance metrics for different endpoints and an aggregated view.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/health	11353	0	17	22	37	17.54	12	203	60.15	26.7	0
GET	/products/search	44550	0	18	520	30000	456.9	12	30128	1469.09	116.7	0
Aggregated		55903	0	17	520	600	367.67	12	30128	1182.96	143.4	0

⚠ The Deceptive 0%

Requests are NOT being rejected — they're just taking 30 seconds each.

From a user perspective, this IS a failure.

SLOs based on error rate alone miss this completely. You need P95/P99.

Fail Fast

500ms context timeout — don't wait for a broken dependency

```
fixed/main.go

ctx, cancel := context.WithTimeout(
    ctx, 500*time.Millisecond)
defer cancel()

result, err := aiRankerWithTimeout(ctx, products)
if err != nil {
    return products, "timeout_fallback" // ✓ FAIL FAST
}
```

How It Works

- Set 500ms deadline on every aiRanker call
- Inner goroutine calls ranker asynchronously
- select {} races: ranker result vs. timer
- Timer fires first → return unranked results
- Goroutine freed immediately — no 30s block

Before: **30,000ms hang**



After: **500ms max**

User still gets a valid response — just without AI ranking. Graceful degradation.

Circuit Breaker

Stop calling a known-broken service

CLOSED

Normal operation
Call ranker

CloudWatch Logs — seen live during experiment:

[circuit breaker] OPEN after 6 failures

[fail fast] ranker timed out: returning unranked results

[circuit breaker] CLOSED: ranker service recovered

The screenshot shows the AWS CloudWatch console interface. The breadcrumb navigation indicates the path: CloudWatch > Log management > /ecs/CS6650L2 > ecs/CS6650L2-container/7228247951a84b118ea23cc038c16c72. The left sidebar shows the 'Logs' section expanded, with 'Log Management' selected. The main panel displays 'Log events' for the specified log group. A filter bar at the top allows searching for events. Below the filter bar, a table lists log events with columns for 'Timestamp' and 'Message'. The events show a sequence of failures and the circuit breaker opening, followed by a 'fail fast' event where the ranker timed out, and finally a 'CLOSED' event indicating the ranker service recovered.

CloudWatch

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom Local timezone Display

Timestamp	Message
2026-02-27T22:24:29.517-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 6 failures - skipping ranker for 10s
2026-02-27T22:24:29.517-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.555-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 7 failures - skipping ranker for 10s
2026-02-27T22:24:29.555-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.570-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 8 failures - skipping ranker for 10s
2026-02-27T22:24:29.570-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.742-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 9 failures - skipping ranker for 10s
2026-02-27T22:24:29.742-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.752-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 10 failures - skipping ranker for 10s
2026-02-27T22:24:29.752-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.882-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 11 failures - skipping ranker for 10s
2026-02-27T22:24:29.882-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results
2026-02-27T22:24:29.897-08:00	2026/02/28 06:24:29 [circuit breaker] OPEN after 12 failures - skipping ranker for 10s
2026-02-27T22:24:29.897-08:00	2026/02/28 06:24:29 [fail fast] ranker timed out: ranker timeout: returning unranked results

No newer events at this moment. Auto retry paused. [Resume](#)

Bulkhead

Limit concurrent callers — protect the goroutine pool

The Nightclub Analogy

Only 10 people allowed inside at once.

If 10 are already in, the bouncer turns you away immediately — no queue.

You don't wait indefinitely. You go home now.

In your server:

Max 10 goroutines calling aiRanker at once.

11th request → instant rejection → unranked results in ~18ms

```
// Semaphore (buffered channel)
bulkhead = make(chan struct{}, 10)

if !bulkhead.Acquire() {
    return products, "bulkhead_rejected"
}
```

Live Result from Experiment

```
--region us-west-2
• (base) emilychen@emilys-Air-2 terraform % curl http://CS6650L2-alb-1653736226.us-west-2.elb.amazonaws.com/health
{"bulkhead_rejected":"433","circuit_state":"0","status":"ok"}
⦿ (base) emilychen@emilys-Air-2 terraform % watch -n 2 curl -s http://CS6650L2-alb-1653736226.us-west-2.elb.amazonaws.com/health
```

433 requests that would have blocked goroutines were instead served in ~18ms each — without touching the broken ranker at all.

Three Layers, One Guard

Each pattern covers the gap the previous one leaves

```
func callRankerSafely(...) {  
    // Layer 1: Bulkhead  
    if !bulkhead.Acquire() {  
        return "bulkhead_rejected"  
    }  
  
    // Layer 2: Circuit Breaker  
    if !breaker.Allow() {  
        return "circuit_open"  
    }  
  
    // Layer 3: Fail Fast  
    ctx := WithTimeout(500ms)  
    result, err := aiRanker(ctx)  
    return "ranked" // or "timeout_fallback"  
}
```

Fail Fast

✓ Stops individual goroutines blocking for 30s

→ Still wastes 500ms when service is totally down

Circuit Breaker

✓ Stops calling a known-broken service entirely

→ Window before 5 failures — goroutines still pile up

Bulkhead

✓ Caps goroutines that can wait, even during that window

→ —

```
ranker_status: "ranked" | "timeout_fallback" | "circuit_open" |  
"bulkhead_rejected"
```

SCENE 3 · RECOVERY

The Recovered System

Same load, same infrastructure
— 3 patterns deployed

 LOCUST

Host

http://CS6650L2-alb-1653736226.us-west-2.elb.ama...

Status

RUNNING

Users

50

RPS

143.4

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

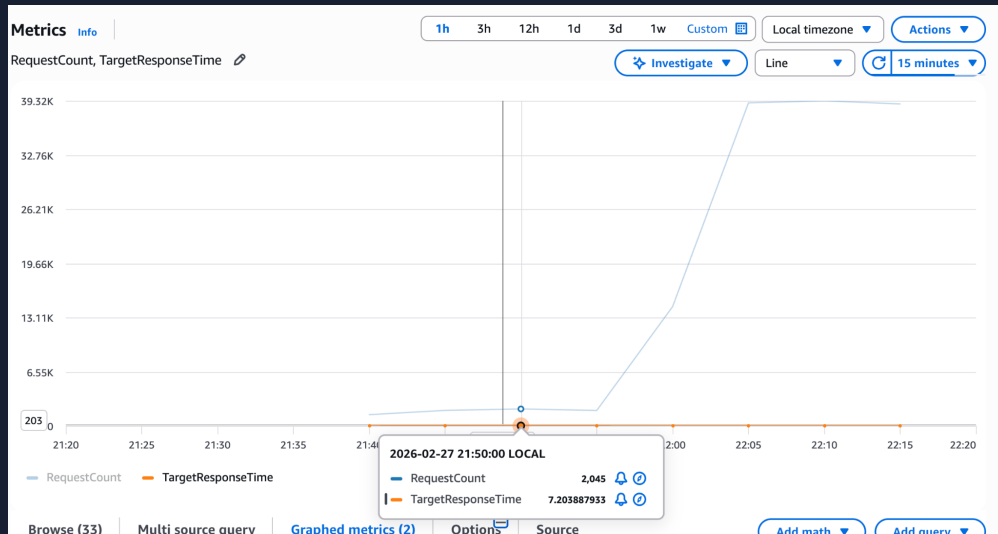
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/health	11353	0	17	22	37	17.54	12	203	60.15	26.7	0
GET	/products/search	44550	0	18	520	30000	456.9	12	30128	1469.09	116.7	0
	Aggregated	55903	0	17	520	600	367.67	12	30128	1182.96	143.4	0

1.3 RPS → 143.4 RPS · 110× throughput improvement · Zero failures

Metric	Broken ❌	Fixed ✅
Throughput (RPS)	1.3	143.4
P95 Latency	30,000 ms	520 ms
P99 Latency	30,000 ms	600 ms
Avg Latency	7,982 ms	457 ms
ALB Req / 15min	~2,045	~39,320
ALB Avg Response	7.2 seconds	~0.1 seconds
Bulkhead Rejected	—	433+ requests

CloudWatch: The Cliff

ALB metrics confirm the transition independently of Locust



At 21:50 (broken)

RequestCount: 2,045

After 22:00 (fixed)

RequestCount: 39,320

19× more requests served with 72× lower latency — measured independently by AWS at the load balancer level.

CS6650 · Emily Chen · Feb 2026

What This Taught Me

Three takeaways that go beyond the assignment

01

Error rate is a lie

Broken version: 0% errors, 30,000ms P99. The server was functionally dead while Locust reported everything green. P95/P99 are non-negotiable in any real SLO.

02

Patterns are layered, not swapped

Fail fast alone still wastes 500ms on a totally dead service. Circuit breaker alone has a window before it opens. Each pattern fills the gap the previous one creates.

03

Expose your degradation mode

ranker_status: "circuit_open" | "bulkhead_rejected" in the response body means you can diagnose partial failures without digging through logs. Design observability in.

Project Proposal

Coming next — check Piazza for the full post

Note History No history yet

   note @835 

Project Proposal

Updated 4 days ago by Shixian

Project Title: Distributed ML Inference Server with Fault-Tolerant Load Balancing

Codebase Description

I am developing a distributed inference serving system in Go that explores the intersection of high-availability systems and ML model deployment. While most ML projects focus on model accuracy, this project focuses on the **control plane and data plane challenges** of serving models at scale — specifically, how a system routes, balances, and recovers from failures under real request load.

The system architecture has three components:

Load Balancer (Go): A centralized orchestrator using `httputil.ReverseProxy` to manage traffic distribution across the worker pool. It implements two routing strategies — round-robin and least-connections — which can be toggled to compare behavior under different workload conditions. A front-end request buffer absorbs traffic bursts that exceed aggregate worker throughput, preventing cascading failures under overload.

Worker Nodes (Go): Independent HTTP servers that each load a pre-trained MNIST classifier at startup and serve predictions via a simple inference endpoint. Workers are stateless beyond the model, making horizontal scaling straightforward — adding a worker requires no coordination beyond registering with the load balancer.

Health & Membership Layer: A failure detector that uses periodic active polling to maintain an up-to-date view of cluster state. When a worker fails to respond within a configurable threshold, the load balancer removes it from the routing pool and redistributes traffic to healthy nodes. The polling interval is intentionally tunable to support the fault tolerance experiment.

Scalability Experiments

Experiment 1 — Throughput Scaling & Bottleneck Analysis (Strong Scaling)

Using Locust to generate sustained inference request load, I will incrementally scale workers from 1 → 2 → 4 and measure requests/sec and p99 latency at each level. The goal is to plot an empirical speedup curve and identify the "knee of the curve" — the point where the load balancer's own CPU and network overhead becomes the primary bottleneck rather than worker capacity. This directly evaluates the horizontal scalability of the serving layer.

Experiment 2 — Load Balancing Strategy Under Workload Imbalance

I will introduce artificial latency variance across request types to simulate heterogeneous inference tasks (e.g., some inputs routed to a "slow" handler, others to a "fast" one). Under this skewed workload, I will compare round-robin vs. least-connections on p99 latency and worker utilization variance. The hypothesis is that least-connections outperforms round-robin when request durations are non-uniform, and the experiment will quantify the crossover point — directly evaluating the system's ability to handle noisy neighbor effects.